

Nhóm: 1

Thành viên: Tô Thành Nguyên

Ngày báo cáo: 23/8/2026

Những nội dung đã làm trong 2 tuần qua:

Sprint 1: Nghiên cứu, thiết kế kiến trúc và vòng lặp Agent tối thiểu (10/08 - 23/08)

1. Khởi tạo dự án Python, yêu cầu phiên bản Python từ 3.11 trở lên và tổ chức package laplace.
2. Xây dựng lớp cấu hình ứng dụng bằng pydantic-settings, sử dụng tiền tố biến môi trường LAPLACE_.
3. Tạo tệp cấu hình mẫu .env.example và bổ sung quy tắc loại trừ các tệp chứa thông tin nhạy cảm.
4. Tạo entrypoint python -m laplace, hỗ trợ kiểm tra trạng thái môi trường và chạy nhiệm vụ mẫu từ dòng lệnh.
5. Xây dựng vòng lặp Agent tuần tự cho nhiệm vụ mẫu, có giới hạn số bước để tránh chạy vô hạn.
6. Bổ sung các trạng thái kết thúc gồm completed, failed và step_limit.
7. Bổ sung lý do kết thúc bằng tiếng Việt cho từng trạng thái của Agent.
8. Kiểm tra dữ liệu đầu vào sai định dạng tại domain boundary và dừng ngay khi một bước thực thi thất bại.
9. Thiết lập kiểm thử tự động bằng pytest và kiểm tra chất lượng mã nguồn bằng Ruff.
10. Thiết lập GitHub Actions để chạy kiểm thử trên Python 3.11 và Python 3.12.
11. Cập nhật README với hướng dẫn cài đặt môi trường, chạy chương trình và kiểm tra chất lượng mã nguồn.
12. Kết quả tại checkpoint cuối Sprint 1: Ruff đạt, 13 bài kiểm thử đạt, CLI mặc định và CLI nhiệm vụ mẫu chạy thành công.

Những nội dung sẽ làm trong 2 tuần tới:

1. Hoàn thiện lớp lưu trữ dữ liệu bằng SQLAlchemy 2.0 và SQLite.
2. Xây dựng các bảng chính gồm người dùng, hội thoại, tin nhắn, tác vụ, bước thực thi, lời gọi công cụ, lời gọi mô hình và trace.
3. Hoàn thiện nhật ký từng bước của Agent để lưu quyết định, công cụ được sử dụng, kết quả và lý do dừng.
4. Xây dựng Telegram Bot bằng aiogram 3.x với các lệnh cơ bản, xử lý tin nhắn và tệp đính kèm.
5. Gắn người dùng Telegram với phiên làm việc riêng và kiểm tra dữ liệu của các người dùng không bị lẫn với nhau.
6. Xử lý phản hồi dài, trạng thái đang xử lý, cập nhật tiến độ từng bước và yêu cầu hủy lượt đang chạy.
7. Cài đặt giới hạn tần suất yêu cầu theo từng người dùng.
8. Kết nối mô hình ngôn ngữ thật và cho phép thay đổi Gemini, OpenAI hoặc Groq bằng cấu hình.
9. Ghi nhận lượng token, chi phí và độ trễ của từng lời gọi mô hình.

10. Xây dựng system prompt, schema hành động và kiểm tra tham số công cụ bằng Pydantic.
11. Thực hiện kiểm thử toàn tuyến từ Telegram đến mô hình ngôn ngữ và phản hồi cho người dùng.

Những trở ngại: